

# Problem – 8 (STUDENT EXCUSE GENERATOR)

## 1. Basic starting prompt:

Write a beginner-friendly C++ console program that generates random student excuses.

Requirements:

- Ask the user to enter their name
- Store at least 10 excuse templates in a vector
- Each template should include a placeholder {name}
- Randomly select one excuse each time the program runs
- Replace {name} with the user input
- Display the final excuse

Add comments in the code so that a first-year student can easily understand it.

## Response:

```
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib> // for rand() and srand()
#include <ctime>   // for time()

using namespace std;

int main() {
    // Step 1: Ask user to enter their name
    string name;
    cout << "Enter your name: ";
    getline(cin, name); // allows full name with spaces

    // Step 2: Store at least 10 excuse templates in a vector
    vector<string> excuses = {
        "{name} couldn't finish the assignment because the laptop updated for hours.",
        "{name} was about to complete the homework when the Wi-Fi disappeared.",
        "{name}'s pet accidentally sat on the keyboard and deleted everything.",
        "{name} got stuck helping a friend with their project all night.",
        "{name} forgot to save the file and the system crashed.",
        "{name} had a sudden power cut right before submission.",
        "{name} was researching too much and lost track of time.",
        "{name} had to attend an unexpected family function.",
        "{name} was feeling unwell and couldn't concentrate on work.",
        "{name} tried finishing the assignment but the software stopped working."
    };

    // Randomly select an excuse
    srand(time(0));
    int index = rand() % excuses.size();
    string excuse = excuses[index];

    // Replace {name} with the user input
    excuse.replace(excuse.find("{name}", 0), 7, name);

    // Display the final excuse
    cout << excuse << endl;
};
```



```

// Step 3: Initialize random seed
// This ensures different random output each time program runs
srand(time(0));

// Step 4: Generate a random index
int randomIndex = rand() % excuses.size();

// Step 5: Pick a random excuse
string selectedExcuse = excuses[randomIndex];

// Step 6: Replace "{name}" with actual user input
// find() returns position of substring
size_t pos = selectedExcuse.find("{name}");
while (pos != string::npos) {
    selectedExcuse.replace(pos, 6, name); // 6 = length of "{name}"
    pos = selectedExcuse.find("{name}");
}

// Step 7: Display the final excuse
cout << "\nGenerated Excuse:\n";
cout << selectedExcuse << endl;

return 0;
}

```

### 💡 How it works (simple explanation):

- You enter your name.
- The program stores multiple excuses in a list (vector).
- It randomly picks one excuse.
- It replaces `{name}` with your actual name.
- Then it prints the final excuse.

## 2. Improve creativity prompt:

Improve the previous C++ program by making the excuses more creative and funny.

- Add at least 5 more unique excuse templates
- Keep the templates simple but humorous
- Make sure each excuse uses the `{name}` placeholder

## Response :

```

#include <iostream>
#include <vector>
#include <string>
#include <cstdlib> // for rand() and srand()
#include <ctime>    // for time()

using namespace std;

int main() {
    // Step 1: Ask user to enter their name
    string name;
    cout << "Enter your name: ";
    getline(cin, name);

    // Step 2: Store funny excuse templates
    vector<string> excuses = {
        "{name} couldn't finish the assignment because the laptop updated for 7 hours straight.",
        "{name} was about to submit homework when the Wi-Fi went on vacation.",
        "{name}'s pet sat on the keyboard and wrote its own assignment instead.",
        "{name} opened the assignment but accidentally opened Netflix too.",
        "{name} forgot to save the file and the computer decided to restart itself.",
        "{name} had a power cut right at the most important moment.",
        "{name} was researching so deeply that time forgot about {name}.",
        "{name} had to attend a surprise family function (even {name} was surprised).",
        "{name} wasn't feeling well... mainly because of too much procrastination.",
        "{name} tried finishing the assignment but the software said 'Not today'."
    };

```

```

// New funny excuses 🐼
"{name}'s brain entered airplane mode and refused to come back.",
"{name} was attacked by a sudden wave of sleep during study time.",
"{name} thought today was Sunday... turns out it was not.",
"{name} waited for motivation to arrive, but it got lost on the way.",
"{name} accidentally started cleaning the room instead of studying.",
"{name} spent 2 hours choosing the perfect pen to start the assignment.",
"{name} opened the book and immediately needed a snack break.",
"{name} was about to start, but then remembered something unimportant to do.",
"{name} tried to focus, but the ceiling fan became very interesting.",
"{name} was debugging life instead of the assignment."
};

```

```

// Step 3: Initialize random seed
srand(time(0));

// Step 4: Generate random index
int randomIndex = rand() % excuses.size();

// Step 5: Pick random excuse
string selectedExcuse = excuses[randomIndex];

// Step 6: Replace {name} with user name
size_t pos = selectedExcuse.find("{name}");
while (pos != string::npos) {
    selectedExcuse.replace(pos, 6, name);
    pos = selectedExcuse.find("{name}");
}

// Step 7: Display result
cout << "\nGenerated Excuse:\n";
cout << selectedExcuse << endl;

return 0;
}

```

### 🌟 What's improved:

- Added 10+ new funny excuses
- More relatable student humor
- Some excuses use `{name}` multiple times for better effect

## 3. Bug fixing prompt:

I ran the program and noticed a problem.

The placeholder `{name}` is not being replaced correctly in some cases.

Fix the code so that all occurrences of `{name}` are replaced properly in every excuse.

Also explain what the issue was and how you fixed it.

## Response :

Good catch — this is exactly the kind of bug beginners run into 🙌

### 🔍 What was the issue?

Your logic *almost* works, but the problem happens because:

- You were calling `find("{name}")` again **from the beginning of the string each time**
- After replacing `{name}` with the actual name, the string length changes
- This can sometimes cause:
  - Skipping occurrences
  - Re-checking already replaced parts (especially if name contains similar text)

So in some cases, not all `{name}` placeholders were replaced correctly

### ✅ How we fix it

We fix this by:

- Continuing the search **from the position after the replacement**
- This ensures we don't miss any occurrences or loop incorrectly

### ✅ Fixed Code (Important Part Highlighted)

```
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {
    string name;
    cout << "Enter your name: ";
    getline(cin, name);

    vector<string> excuses = {
        "{name} couldn't finish the assignment because the laptop updated for 7 hours straight.",
        "{name} was about to submit homework when the Wi-Fi went on vacation.",
        "{name}'s pet sat on the keyboard and wrote its own assignment instead.",
        "{name} opened the assignment but accidentally opened Netflix too.",
        "{name} forgot to save the file and the computer restarted.",
        "{name} was researching so deeply that time forgot about {name}.",
        "{name} waited for motivation, but it never showed up for {name}.",
        "{name} thought today was Sunday... but reality disagreed.",
        "{name} tried to study, but the ceiling fan became too interesting.",
        "{name} was debugging life instead of the assignment."
    };

    srand(time(0));
    int randomIndex = rand() % excuses.size();
```

```
string selectedExcuse = excuses[randomIndex];

// ✅ FIXED REPLACEMENT LOGIC
size_t pos = 0;
while ((pos = selectedExcuse.find("{name}", pos)) != string::npos) {
    selectedExcuse.replace(pos, 6, name);
    pos += name.length(); // move forward after replacement
}

cout << "\nGenerated Excuse:\n";
cout << selectedExcuse << endl;

return 0;
}
```

### 🧠 Simple Understanding

Think of it like:

- Old way: "Start searching from beginning again and again" ❌
- New way: "Continue searching from where you left off" ✅

## 4. Random issue fix (second bug):

The program is giving the same excuse again and again when I run it multiple times.

Fix this issue so that different excuses are generated each time the program runs.

Explain the reason behind the problem.

This is a classic C++ randomness issue — and it's not your fault, it's how `rand()` works 🙌

### 🔍 Why are you getting the same excuse again and again?

The problem is this line:

```
</> C++  
srand(time(0));
```

#### ! What's happening:

- `time(0)` returns time in seconds
- If you run the program multiple times **within the same second**, the seed value is the same
- Same seed → same random sequence → same excuse 😞

#### ✅ How to fix it

We need a **better random generator** that doesn't depend on seconds.

#### ✓ Best modern fix (C++11 and above):

Use `<random>` library instead of `rand()`



#### ✅ Fixed & Improved Code (Recommended)

```

#include <iostream>
#include <vector>
#include <string>
#include <random>    // modern random library

using namespace std;

int main() {
    string name;
    cout << "Enter your name: ";
    getline(cin, name);

    vector<string> excuses = {
        "{name} couldn't finish the assignment because the laptop updated for 7 hours straight.",
        "{name} was about to submit homework when the Wi-Fi went on vacation.",
        "{name}'s pet sat on the keyboard and wrote its own assignment instead.",
        "{name} opened the assignment but accidentally opened Netflix too.",
        "{name} forgot to save the file and the computer restarted.",
        "{name} was researching so deeply that time forgot about {name}.",
        "{name} waited for motivation, but it never showed up for {name}.",
        "{name} thought today was Sunday... but reality disagreed.",
        "{name} tried to study, but the ceiling fan became too interesting.",
        "{name} was debugging life instead of the assignment."
    };

    // ✅ Better random generator
    random_device rd;          // gets random seed from system
    mt19937 gen(rd());         // Mersenne Twister engine
    uniform_int_distribution<> dist(0, excuses.size() - 1);

    int randomIndex = dist(gen);

    string selectedExcuse = excuses[randomIndex];

    // Replace all {name}
    size_t pos = 0;
    while ((pos = selectedExcuse.find("{name}", pos)) != string::npos) {
        selectedExcuse.replace(pos, 6, name);
        pos += name.length();
    }

    cout << "\nGenerated Excuse:\n";
    cout << selectedExcuse << endl;

    return 0;
}

```

## 🧠 Simple Understanding

Old Method ❌

rand()

Same output if run quickly

Based on seconds

New Method ✅

<random>

Different output every run

Uses real randomness

## 5.Edge case handling:

Modify the program to handle edge cases.

- If the user enters an empty name, display an error message
- Prevent the program from crashing
- Keep the solution simple and beginner-friendly

## Response :

Good thinking — handling edge cases is what makes your program solid 👍

Here's a **simple and beginner-friendly fix** that:

- Checks if the name is empty
- Prevents invalid input
- Keeps everything easy to understand

```
#include <iostream>
#include <vector>
#include <string>
#include <random>

using namespace std;

int main() {
    string name;

    // Step 1: Ask for user input
    cout << "Enter your name: ";
    getline(cin, name);

    // ✅ Edge Case Handling: Check if name is empty
    if (name.empty()) {
        cout << "Error: Name cannot be empty. Please run the program again.\n";
        return 0; // safely exit program
    }

    // Step 2: Store excuses
    vector<string> excuses = {
        "{name} couldn't finish the assignment because the laptop updated forever.",
        "{name} was about to submit homework when Wi-Fi disappeared.",
        "{name}'s pet pressed random keys and submitted something else.",
        "{name} opened the assignment but got distracted instantly.",
        "{name} forgot to save the file and the system crashed.",
        "{name} was researching so deeply that time forgot about {name}."
    };
```



```

        "{name} waited for motivation... it never came.",
        "{name} thought today was Sunday (it was not).",
        "{name} tried to focus but got distracted by everything.",
        "{name} was debugging life instead of homework."
    };

    // Step 3: Random number generator
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> dist(0, excuses.size() - 1);

    int randomIndex = dist(gen);
    string selectedExcuse = excuses[randomIndex];

    // Step 4: Replace all {name}
    size_t pos = 0;
    while ((pos = selectedExcuse.find("{name}", pos)) != string::npos) {
        selectedExcuse.replace(pos, 6, name);
        pos += name.length();
    }

    // Step 5: Display result
    cout << "\nGenerated Excuse:\n";
    cout << selectedExcuse << endl;

    return 0;
}

```

### 💡 What we fixed (in simple terms)

#### ❌ Problem:

If user presses Enter without typing anything,

`name = ""` (empty string)

This could lead to:

- Weird output like:  
→ " couldn't finish the assignment..."
- Or poor user experience

#### ✅ Solution:

📄 C++

```
if (name.empty())
```

- Checks if input is empty
- Shows error message
- Stops program safely using `return 0;`

## 6. Final enhancement:

Make the final program more user-friendly.

- Add proper headings and spacing in the output
- Make the output look clean and readable
- Do not make the code complex

## Response:

```
#include <iostream>
#include <vector>
#include <string>
#include <random>

using namespace std;

int main() {
    string name;

    // ★ Heading
    cout << "=====\\n";
    cout << "    STUDENT EXCUSE GENERATOR 🤖\\n";
    cout << "=====\\n\\n";

    // Input
    cout << "Enter your name: ";
    getline(cin, name);

    // ✅ Handle empty input
    if (name.empty()) {
        cout << "\\n⚠ Error: Name cannot be empty.\\n";
        cout << "Please run the program again.\\n";
        return 0;
    }

    // Excuses list
    vector<string> excuses = {
        "{name} couldn't finish the assignment because the laptop updated forever.",
        "{name} was about to submit homework when Wi-Fi disappeared.",
```

```

    "{name}'s pet pressed random keys and submitted something else.",
    "{name} opened the assignment but got distracted instantly.",
    "{name} forgot to save the file and the system crashed.",
    "{name} was researching so deeply that time forgot about {name}.",
    "{name} waited for motivation... it never came.",
    "{name} thought today was Sunday (it was not).",
    "{name} tried to focus but got distracted by everything.",
    "{name} was debugging life instead of homework.",
    "{name}'s brain went into power-saving mode.",
    "{name} was about to start but needed a snack break first.",
    "{name} spent more time planning than actually doing the work.",
    "{name} blinked and suddenly it was too late.",
    "{name} got stuck in a loop of '5 more minutes'."
};

// Random generator
random_device rd;
mt19937 gen(rd());
uniform_int_distribution<> dist(0, excuses.size() - 1);

string selectedExcuse = excuses[dist(gen)];

// Replace {name}
size_t pos = 0;
while ((pos = selectedExcuse.find("{name}", pos)) != string::npos) {
    selectedExcuse.replace(pos, 6, name);
    pos += name.length();
}

```

```

// ★ Clean Output
cout << "\n-----\n";
cout << " 🎮 Your Generated Excuse:\n";
cout << "-----\n\n";

cout << "👉 " << selectedExcuse << "\n\n";

cout << "-----\n";
cout << " Thank you for using the generator!\n";
cout << "-----\n";

return 0;
}

```

## 🌟 What's improved

- ✓ Clear heading/title
- ✓ Proper spacing (no clutter)
- ✓ Clean sections with lines
- ✓ Highlighted output using symbols (👉 🎮)
- ✓ Friendly closing message

